

Detecção e Refatoração de Test Smells com ESLint (Jest)

Disciplina: Testes de Software

Trabalho: Detecção e Refatoração de Test Smells

Aluno: Thiago Borges Laass

Matrícula: 836095

Data: 20/05/2026

1. Introdução e Contexto

Este trabalho aborda a relação entre qualidade de testes e a capacidade real de detectar defeitos. Mesmo com boa cobertura, uma suíte pode ser ineficaz quando contém **Test Smells**: sinais de problemas de design nos testes que os tornam **frágeis, difíceis de entender, difíceis de manter** e, em alguns casos, incapazes de falhar quando deveriam.

Para apoiar a detecção e a remoção desses problemas, foi utilizada análise estática com **ESLint** e o plugin **eslint-plugin-jest**, e uma suíte “mal cheirosa” foi refatorada para uma versão “clean”, aplicando boas práticas como o padrão **Arrange, Act, Assert (AAA)**.

2. Análise de Smells

A seguir estão três Test Smells identificados no arquivo `test/userService.smelly.test.js`, com explicação do risco associado.

2.1. Lógica condicional no teste (Conditional Logic / Conditional Expect)

Onde aparece: no teste deve desativar usuários se eles não forem administradores, que usa `for` e `if/else` e executa `expect(...)` apenas em determinados caminhos.

Por que é um smell: testes com fluxo de controle (`if`, `for`) tendem a ficar menos diretos e mais difíceis de ler. Além disso, quando asserções são condicionais, parte das expectativas pode **não ser executada** dependendo dos dados, o que reduz a confiança de que o comportamento foi realmente validado.

Risco:

- esconder regressões em um dos ramos;
- tornar o teste mais difícil de depurar (é menos claro quais asserts rodaram);
- aumentar a complexidade e diminuir a intenção do teste.

2.2. Teste desabilitado (Disabled Test)

Onde aparece: `test.skip('deve retornar uma lista vazia quando não há usuários', ...)`.

Por que é um smell: um teste desabilitado é um “buraco” na suíte. Ele tende a permanecer esquecido e cria uma falsa sensação de completude.

Risco:

- comportamentos importantes deixam de ser exercitados;
- falhas só aparecem em produção;
- a suíte passa sem realmente cobrir cenários relevantes.

2.3. Teste frágil por dependência de formatação (Fragile Test)

Onde aparece: no teste deve gerar um relatório de usuários formatado, que espera um texto altamente específico (inclusive com `\n` e ordem exata).

Por que é um smell: o teste verifica detalhes de *formatação* e não apenas o comportamento. Pequenas mudanças legítimas no relatório (por exemplo, adicionar um espaço, trocar a ordem dos campos ou ajustar o cabeçalho) podem quebrar o teste sem que exista um defeito real.

Risco:

- aumento de falsos positivos (testes quebram por mudanças cosméticas);
- manutenção excessiva;
- incentivo a “fixar” o formato por medo de quebrar testes, prejudicando evolução do sistema.

3. Processo de Refatoração (Antes e Depois)

O teste mais problemático escolhido foi o de desativação de usuários, por conter loop, condicional e asserções executadas de forma condicional.

3.1. Antes (arquivo smelly)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40,
true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  // O teste tem um loop e um if, tornando-o complexo e menos claro.
  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      // Este expect só roda para o usuário comum.
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      // E este só roda para o admin.
```

```

        expect(resultado).toBe(false);
    }
}
});

```

Problemas:

- viola AAA (mistura arranjo, ação e asserção dentro do loop);
- usa `for` e `if/else` (complexidade desnecessária para um teste);
- `expect` é condicional (detectado por `jest/no-conditional-expect`).

3.2. Depois (arquivo clean)

A refatoração separou o cenário em **dois testes focados**, cada um validando um único comportamento de forma direta e sem condicionais.

```

test('deactivateUser desativa um usuário comum e atualiza o status', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(usuarioAtualizado).toEqual(
    expect.objectContaining({
      id: usuarioComum.id,
      status: 'inativo',
    })
  );
});

test('deactivateUser não desativa um usuário administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
  expect(usuarioAtualizado).toEqual(
    expect.objectContaining({
      id: usuarioAdmin.id,
      isAdmin: true,
      status: 'ativo',
    })
  );
});

```

Decisões e melhorias obtidas:

- **AAA explícito** em todos os testes, aumentando legibilidade;
- **remoção total** de `for` e `if/else` nos testes;

- asserções passam a ser **sempre executadas** (sem `expect` condicional);
- cenários ficam mais claros (um teste = um comportamento).

3.3. Outras melhorias relevantes

- O caso “menor de idade” deixou de usar `try/catch` (que poderia passar silenciosamente se não lançasse erro) e passou a usar `toThrow(...)`, garantindo falha quando a exceção não ocorrer.
- O relatório de usuários passou a ser validado com `toContain(...)` para cabeçalho e dados essenciais (IDs e nomes), reduzindo fragilidade por mudanças cosméticas.

4. Relatório da Ferramenta (ESLint)

4.1 Saída do ESLint

Abaixo está a saída obtida na análise do teste “smelly”, evidenciando problemas como `expect` condicional e teste desabilitado:

```
/home/thiago/repos/test-smells/__tests__/userService.smelly.test.js
44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-
expect
46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-
expect
49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-
expect
73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-
expect
77:3  warning  Disabled test                          jest/no-disabled-tests
77:3  warning  Test has no assertions                  jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

4.3. Como a ferramenta automatizou a detecção

O ESLint, com regras específicas do Jest, automatizou a identificação de padrões ruins que são fáceis de “passar batido” em revisão manual. Em especial:

- `jest/no-conditional-expect` destaca testes que podem executar asserts apenas em alguns caminhos, reduzindo confiabilidade.
- `jest/no-disabled-tests` aponta testes desativados que enfraquecem a suíte.
- `jest/expect-expect` ajuda a identificar testes sem asserções, que podem se tornar “falsos positivos”.

5. Conclusão

A refatoração da suíte evidenciou que “testes passando” não é sinônimo de “testes bons”. Ao remover lógica condicional, separar cenários e aplicar AAA, os testes ficaram mais diretos, menos

frágeis e mais alinhados ao comportamento esperado do sistema.

Ferramentas de análise estática como ESLint (com plugins especializados) atuam como um **guardrail**: reforçam padrões de escrita, reduzem o risco de smells voltarem a aparecer e ajudam a manter a suíte sustentável ao longo do tempo. Em conjunto, testes limpos e linting automatizado elevam a confiança no processo de evolução do software e na prevenção de regressões.